

REPORT: PROGRAMMING INTERGRATION PROJECT

KNOWLEDGE RETRIEVAL AND QA FOR COMPUTER SCIENCE COURSES USING RAG

SUPERVISOR(s): Dr. Nguyen Hua Phung

Student: Nguyen Quoc Hieu - 2352344

Computer Science
HCMC University of Technology, Viet Nam

09, 2025



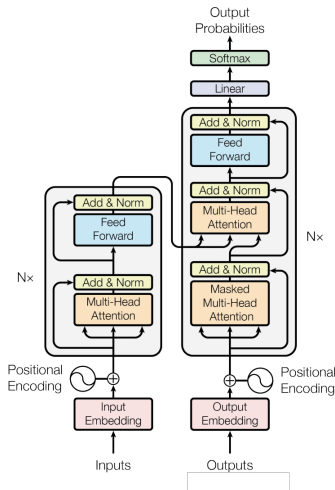
Table of Contents

- 1 Motivation
- 2 Overview
 - Data Preprocessing
 - Embedding
 - Vector DB
 - Retrieval
 - Generation (QA)
- 3 Another Idea
- 4 Summary



BERT

Encoder



GPT

Decoder

Table of Contents

1 Motivation

2 Overview

- Data Preprocessing
- Embedding
- Vector DB
- Retrieval
- Generation (QA)

3 Another Idea

4 Summary



Motivation

My ideas about implementing RAG-based project combining with Encoders and Decoders:

- Computer Science students need to go through a large amount of lecture slides and textbooks.



Motivation

My ideas about implementing RAG-based project combining with Encoders and Decoders:

- Computer Science students need to go through a large amount of lecture slides and textbooks.
- This process is time-consuming and inefficient.



Motivation

My ideas about implementing RAG-based project combining with Encoders and Decoders:

- Computer Science students need to go through a large amount of lecture slides and textbooks.
- This process is time-consuming and inefficient.
- Idea: integrate all course materials into a **RAG-based chatbot**.



My ideas about implementing RAG-based project combining with Encoders and Decoders:

- Computer Science students need to go through a large amount of lecture slides and textbooks.
- This process is time-consuming and inefficient.
- Idea: integrate all course materials into a **RAG-based chatbot**.
 - Quickly provide answers about specific courses or topics.
 - Facilitate access to necessary information in each CS subject.
 - Summarize materials and generate cheatsheets from lecture slides.



My ideas about implementing RAG-based project combining with Encoders and Decoders:

- Computer Science students need to go through a large amount of lecture slides and textbooks.
- This process is time-consuming and inefficient.
- Idea: integrate all course materials into a **RAG-based chatbot**.
 - Quickly provide answers about specific courses or topics.
 - Facilitate access to necessary information in each CS subject.
 - Summarize materials and generate cheatsheets from lecture slides.
- **Goal:** Improve study efficiency and provide an interactive learning assistant.



Table of Contents

1 Motivation

2 Overview

- Data Preprocessing
- Embedding
- Vector DB
- Retrieval
- Generation (QA)

3 Another Idea

4 Summary



General Architecture:

- **Data sources:** PDF/PPTX slides + syllabus + course codes.



General Architecture:

- **Data sources:** PDF/PPTX slides + syllabus + course codes.
- **Preprocess:** extract text, clean it, chunk (300–500 tokens, 50–100 overlap), attach metadata (course, chapter, page, file, instructor).



General Architecture:

- **Data sources:** PDF/PPTX slides + syllabus + course codes.
- **Preprocess:** extract text, clean it, chunk (300–500 tokens, 50–100 overlap), attach metadata (course, chapter, page, file, instructor).
- **Embedding:** SBERT (sequence-BERT) (e.g., all-MiniLM-L6-v2 or all-mpnet-base-v2).



General Architecture:

- **Data sources:** PDF/PPTX slides + syllabus + course codes.
- **Preprocess:** extract text, clean it, chunk (300–500 tokens, 50–100 overlap), attach metadata (course, chapter, page, file, instructor).
- **Embedding:** SBERT (sequence-BERT) (e.g., all-MiniLM-L6-v2 or all-mpnet-base-v2).
- **VectorDB:** FAISS (local, simple) → optionally upgrade to Qdrant/Chroma.



General Architecture:

- **Data sources:** PDF/PPTX slides + syllabus + course codes.
- **Preprocess:** extract text, clean it, chunk (300–500 tokens, 50–100 overlap), attach metadata (course, chapter, page, file, instructor).
- **Embedding:** SBERT (sequence-BERT) (e.g., all-MiniLM-L6-v2 or all-mpnet-base-v2).
- **VectorDB:** FAISS (local, simple) → optionally upgrade to Qdrant/Chroma.
- **Retrieval:** top-k ($k = 4-8$) + optional re-ranking.



General Architecture:

- **Data sources:** PDF/PPTX slides + syllabus + course codes.
- **Preprocess:** extract text, clean it, chunk (300–500 tokens, 50–100 overlap), attach metadata (course, chapter, page, file, instructor).
- **Embedding:** SBERT (sequence-BERT) (e.g., all-MiniLM-L6-v2 or all-mpnet-base-v2).
- **VectorDB:** FAISS (local, simple) → optionally upgrade to Qdrant/Chroma.
- **Retrieval:** top-k ($k = 4-8$) + optional re-ranking.
- **Generation (QA):** GPT/LLaMA (decoder) with prompts that require citations.



Table: Metadata Sample

Course Code	CO3005
Course Name	Principles of Programming Languages
Instructor	Dr. X
Source File	Lexical Analysis.pdf
Page	12
Section	Syntax
Slide Title	Lexical Analysis Roles
Syllabus URL	(empty)
Created At	2016-08



Embedding Process

- Cleaned text is split into 300–500 token chunks (overlap 50–100).
- Each chunk is encoded with **Sentence-BERT (SBERT)** into a dense vector.
- Store vector + metadata (course, file, page, section).

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode("PPL introduces syntax and semantic")
print(len(emb))
```



- Index embeddings with **FAISS/Qdrant/Chroma**.
- Support filters (e.g., `course = PPL, week = 3`).
- Persist metadata: `{course_code, source_file, page, section}`.

Why it matters: fast nearest-neighbor search, scalable storage, and precise scoping by course/week.



- **Dense search:** cosine similarity, top- k (4–8).

$$\text{TopK}(q, D, k) = \arg \max_{d \in D} \text{top-}k s(q, d).$$

- **Re-ranking** via cross-encoder for better ordering.
- Build prompt with selected chunks and require citations (e.g., (PPL:Lecture03.pdf:12)).
- Generate answer with GPT/LLaMA; keep temperature low to reduce hallucinations.

Outputs: concise answer + source citations for traceability.



Generation (QA)

- Build a strict prompt from **user query** + **top- k contexts**.
- Instruct the LLM to **use only the provided context** and **cite sources** (e.g., (PPL:data-types-handout1.pdf:12)).
- Generate with **GPT/LLaMA**; keep temperature low; limit length.
- Post-process: format answer, validate citations, safety checks.



Table of Contents

1 Motivation

2 Overview

- Data Preprocessing
- Embedding
- Vector DB
- Retrieval
- Generation (QA)

3 Another Idea

4 Summary



Receipt Recognition (YOLO + PaddleOCR: DBNet, SVTR): [Repo](#)

- Built an end-to-end receipt OCR pipeline: YOLO for receipt detection, DBNet for text detection, SVTR for text recognition; benchmarked on SROIE (ICDAR 2019).
- Implemented pre/post-processing (denoise, perspective correction, cropping; regex/heuristic field matching) and exported structured results (JSON/CSV).
- Exposed a PySide6 GUI.
- Added evaluation scripts (precision/recall/F1, CER/WER) and ablation studies; optimized throughput via batching and async I/O.



Table of Contents

1 Motivation

2 Overview

- Data Preprocessing
- Embedding
- Vector DB
- Retrieval
- Generation (QA)

3 Another Idea

4 Summary



Summary

What are still in your mind?

